

Base Station Operation Log — Source File Requirements

For: Datum device / companion-app engineering **From:** CBMI Capture Universe — Base Station provenance team **Status:** Requirements for implementation **Consumes into:**

`base_station_score` → *Data Completeness & Integrity* building block (Stage 1, PPK workflow)

1. Why this file exists (read this first)

The base station already produces a RINEX file. RINEX records **what the receiver observed**. It does **not** record **what the device did**. When a session fails, those two stories diverge, and the gap is invisible in RINEX alone.

The specific failure we cannot otherwise catch: a RINEX file can be perfectly well-formed right up to the point where it was silently truncated — battery died, device was switched off before the log finished, unexpected reset. PPK software accepts the file and produces a degraded solution with no warning. The operator finds out hours later, after a long drive home, or never.

This Operation Log is the **only** source that can certify a session ended cleanly and had adequate power. It exists to answer one question: *"Is this RINEX file actually complete, or does it just look complete?"*

Scope discipline (important): This file must capture **only** completeness and power certification. Do **not** add recording start/end as primary timestamps, gap detection, epoch counts, satellite counts, or signal quality — all of that is already derived from RINEX, and duplicating it creates two sources that can disagree. If a proposed field can be answered from RINEX, it does not belong here.

2. Format

- **File type:** JSON, UTF-8.
- **Structure:** one JSON object per recording session.
- **Delivery:** exported from the Datum device or companion app at the same time as the RINEX download, as part of the upload bundle.
- **Naming:** `operation_log.json` (or `<session_id>_operation_log.json` if multiple sessions per upload).

JSON keeps this consistent with the existing `user_input.json` source file and is trivial to parse on ingest.

3. Required fields

All fields are **nullable**. A `null` means "the device did not report this" and is scored as

unconfirmed — which is deliberately different from a bad value. Never substitute a default (e.g. do not write `0` or `100` for an unknown battery level); write `null`. This lets the score distinguish *known-bad* from *unknown*, which is the difference between failing a session and flagging it for review.

Field	Type	Unit / format	Meaning	Why only this file has it
<code>session_completed_normally</code>	boolean null	—	<code>true</code> only if recording was stopped by deliberate operator action (stop button / normal shutdown). <code>false</code> if it ended any other way.	RINEX ends at a timestamp but cannot say <i>why</i> it ended. This is the single most important field.
<code>unexpected_shutdown_count</code>	integer null	count	Number of unexpected shutdowns / resets during the session. <code>0</code> = healthy. <code>≥1</code> drives a flag.	No power/reset event record exists anywhere in RINEX.
<code>battery_start_pct</code>	number null	0-100	Battery level at session start.	RINEX has no power data of any kind.
<code>battery_end_pct</code>	number null	0-100	Battery level at session end.	"
<code>battery_min_pct</code>	number null	0-100	Lowest battery level reached during the session. More important than the endpoints — a session that dipped to 3% mid-flight was at risk even if it recovered.	"

Field	Type	Unit / format	Meaning	Why only this file has it
<code>session_end_utc</code>	string null	ISO-8601 UTC (<code>2026-05-28T14:32:07Z</code>)	When the device believes the session ended.	Used as a cross-check only against the RINEX last-observation time. A disagreement beyond tolerance reveals a silent truncation neither file shows alone.

4. Requested field (closes a known gap — include if cheap)

Field	Type	Unit / format	Meaning
<code>raw_log_download_confirmed</code>	boolean null	—	<code>true</code> once the raw observation (RINEX) file has actually been transferred off the device.

Why ask for it: This directly closes a CRITICAL / VERY-COMMON failure — crew packs up and drives away with the raw log still stranded on the device's internal memory, discovered only at the office. If the app can stamp this `true` at the moment of transfer, the failure converts from an open gap to fully covered. **Do not block the rest of the spec on this** — ship the required fields even if this one needs more work.

5. Edge cases to define explicitly

These decide how the score interprets the file, so the firmware behaviour must be specified, not left implicit:

1. **Abrupt total power loss (device can't write the log).** If the device dies so suddenly it cannot finish writing this file, what happens — is `session_completed_normally` written `false`, or is the **entire file absent**? Both are acceptable to us, but we must know which,

because "file absent" and "file present with `completed_normally = false`" are scored the same (uncertain completion) only if we know the absence is meaningful.

2. `unexpected_shutdown_count` scope. Does the count include the *final* fatal shutdown, or only mid-session recoverable ones? We recommend counting **only mid-session** events here, and letting `session_completed_normally = false` carry the "ended badly" signal — but either is fine if documented.
 3. **Mains / solar units with no internal battery.** The three battery fields must be `null` (not `0`, not `100`) so the score reads "battery not applicable" rather than "battery dead" or "battery full." Confirm the firmware writes `null` in this case.
-

6. Example payloads

Healthy session:

```
json
{
  "session_completed_normally": true,
  "unexpected_shutdown_count": 0,
  "battery_start_pct": 98,
  "battery_end_pct": 71,
  "battery_min_pct": 71,
  "session_end_utc": "2026-05-28T14:32:07Z",
  "raw_log_download_confirmed": true
}
```

Power interruption during session (still finished, but at risk):

```
json
{
  "session_completed_normally": true,
  "unexpected_shutdown_count": 2,
  "battery_start_pct": 64,
  "battery_end_pct": 22,
  "battery_min_pct": 4,
  "session_end_utc": "2026-05-28T13:58:41Z",
  "raw_log_download_confirmed": true
}
```

Silent truncation (device died, file written by a recovery path):

```
json
```

```
{
  "session_completed_normally": false,
  "unexpected_shutdown_count": 1,
  "battery_start_pct": 40,
  "battery_end_pct": null,
  "battery_min_pct": 2,
  "session_end_utc": "2026-05-28T12:11:00Z",
  "raw_log_download_confirmed": false
}
```

Solar/mains unit (battery not applicable):

```
json
{
  "session_completed_normally": true,
  "unexpected_shutdown_count": 0,
  "battery_start_pct": null,
  "battery_end_pct": null,
  "battery_min_pct": null,
  "session_end_utc": "2026-05-29T17:45:12Z",
  "raw_log_download_confirmed": true
}
```

7. How the score uses each field (so the values are testable)

Field	Drives	Behaviour
<code>session_completed_normally = false</code>	integrity sub-score	Major penalty + <code>FLAG: BASE_SESSION_INTERRUPTED</code>
<code>session_completed_normally = null</code>	integrity sub-score	Degrade to <i>unconfirmed</i> (partial), not fail
<code>unexpected_shutdown_count ≥ 1</code>	integrity sub-score	Penalty scaled by count + flag
<code>battery_min_pct</code> low (threshold TBD, ~≤10%)	integrity sub-score	Risk penalty even if session completed
<code>session_end_utc</code> vs RINEX last epoch	truncation cross-check	Disagreement beyond tolerance → <code>FLAG: BASE_RINEX_TRUNCATED</code>

Field	Drives	Behaviour
raw_log_download_confirmed = false null	completeness	FLAG: BASE_LOG_DOWNLOAD_UNCONFIRMED (advisory)

Exact thresholds are owned by the provenance/scoring layer and will be finalised in the chain build; the firmware only needs to supply the fields above, correctly typed and nullable.